iNeuBytes — Virtual Internship Program (VIIP)

Artificial Intelligence — Course ID: AIINB20726

Task 2: Sentiment Analysis using ML and DL

Overview: Build classical baselines, run controlled experiments on both representation and architecture, and produce a comparative report styled like a mini research paper: hypothesis → method → results → analysis.

Fixed experimental controls (apply to the entire task)

- One fixed random seed, set everywhere (NumPy, TensorFlow/Keras, scikit-learn, Python)
- One fixed train / validation / test split, frozen and reused across every model and experiment
- The same preprocessing pipeline reused consistently (any deviation must be stated)
- Every model logged in a master comparison table — no result is valid unless reproducible and compared on the identical split

Part A — Classical Machine Learning (The Baseline)

Objective: Use traditional machine learning to perform sentiment analysis on a text dataset (IMDb Reviews, Twitter Sentiment, Amazon Reviews, or any suitable dataset), establishing the classical baselines every later experiment is measured against.

Requirements — interns must:

- Load a sentiment analysis dataset
- Report the class distribution; if imbalanced, note its effect and rely on F1-score for judging performance
- Clean the text (remove punctuation, lowercase, remove extra whitespace, remove stop words) and tokenize
- Establish and lock the fixed random seed and train/validation/test split (frozen for the entire task)
- Apply TF-IDF vectorization
- Train Logistic Regression and Support Vector Machine (SVM)
- Compare both models

Expected output — interns must report:

- Data preprocessing steps and class distribution
- TF-IDF feature extraction details (vocabulary size, n-gram range used)
- Logistic Regression and SVM results
- Accuracy, Precision, Recall, F1-score for both models
- Research note: inspect 3–5 misclassified examples and form a hypothesis about what kind of language the classical models get wrong (e.g. sarcasm, negation, long-range context) — revisited in Part C

Part B — Controlled Experiments: Representation and Architecture

Objective: Investigate what actually drives performance — text representation and model architecture — through controlled experiments, changing one variable at a time on the same frozen split.

1. Feature representation study (classical side)

- Vary the TF-IDF configuration: unigrams vs. unigrams+bigrams; different vocabulary sizes / min-document-frequency
- Report which representation most improved the classical F1, and why context (bigrams) might help on this dataset

2. Deep learning pipeline (build the LSTM properly)

- Tokenize, convert text to sequences, apply padding (justify your chosen sequence length using the data's length distribution)
- Use an embedding layer; build an LSTM with dropout to reduce overfitting
- Use the same split and seed as Part A

3. Embedding study (does pre-trained knowledge help?)

- Run separately: a trainable embedding from scratch vs. pre-trained embeddings (GloVe or Word2Vec)
- Report whether pre-trained embeddings helped, and why they might matter more when training data is limited (link to Transfer Learning)

4. Regularization & capacity study (deep learning side)

- Vary dropout rate (e.g. 0.0 vs. 0.3 vs. 0.5) and/or LSTM units
- Plot training vs. validation curves for each; diagnose where overfitting begins
- Report which setting best closed the train–val gap without hurting test F1

Rules for fair comparison (all experiments)

- Same random seed and same train/test split as Part A
- Same evaluation metrics across all models (Accuracy, Precision, Recall, F1)
- Each experiment isolates one change so its effect is attributable

Expected output: a single master experiment table — each row = one model/configuration, columns = Accuracy, Precision, Recall, F1, train-val gap, training time — plus a 1–2 sentence insight per experiment group.

Part C — Final Comparison & Evidence-Based Conclusion

Objective: Bring the best classical model and the best LSTM configuration together, and answer the task's central research question with evidence — including the honest possibility that the simpler model wins.

Requirements — interns must:

- Select the best classical model (from Part A/B) and the best LSTM (from Part B)
- Re-test the Part A misclassification hypothesis: does the LSTM correctly handle the sarcasm/negation/long-context cases the classical models missed? Show specific examples

Success threshold: report whether the LSTM matched or exceeded the best classical model, and explain the result either way. If the LSTM does not beat the classical models, explain why with evidence (dataset size too small for deep learning, training time, class imbalance, sequence length). Recognizing when a simpler model wins — and proving it — is a key data science skill and fully acceptable as a result.

Expected output — interns must report and explain:

- Final comparison table: Logistic Regression vs. SVM vs. LSTM (same metrics for all)
- Training vs. validation curves for the LSTM (evidence of overfitting and whether dropout helped)
- Why LSTM is suitable for text data and how sequential learning helps sentiment analysis
- Whether the LSTM improved performance — supported by the misclassified-examples comparison, not just headline accuracy
- The performance-vs-cost trade-off: the LSTM's accuracy gain (if any) versus its added training time and complexity — was it worth it for this dataset?
- Challenges faced during training and a final, evidence-backed verdict on which approach you would deploy and why

Deliverables for Task 2

Deliverable	Description
Python Code	Complete ML and LSTM implementation +

Deliverable	Description
	requirements.txt + README
Preprocessing Explanation	Cleaning, tokenization, stopword removal
Architecture Diagrams	ML pipeline and LSTM architecture
Performance Table	Logistic Regression vs SVM vs LSTM
Confusion Matrix	For all models
Google Doc Report	Complete task explanation